1.2.62

Vivek K Kumar - EE25BTECH11062

March 30, 2026

Question

Question:
The iteration scheme

$$x_{n+1} = \sqrt{a}\left(1 + \frac{3a^2}{x_n^2}\right) - \frac{3a^2}{x_n}, \quad a > 0 \tag{1}$$

converges to the real number

1. $\sqrt{a}$
2. a
3. $a\sqrt{a}$
4. a^2

Solution I

Since this problem is a fixed-point iteration, and we are supposed to find $x = \lim_{n\to\infty} x_n$. So we simply need to find x where

$$x = g(x) \tag{2}$$

$$g(x) = \sqrt{a}\left(1 + \frac{3a^2}{x^2}\right) - \frac{3a^2}{x} \tag{3}$$

On simplification and solving, we get

$$x^3 - \sqrt{a}x^2 + 3a^2x - 3a^2\sqrt{a} = 0 \tag{4}$$

$$x^2(x - \sqrt{a}) + 3a^2(x - \sqrt{a}) = 0 \tag{5}$$

$$(x^2 + 3a^2)(x - \sqrt{a}) = 0 \tag{6}$$

$$\implies x = \sqrt{a} \tag{7}$$

Hence it converges to $\sqrt{a}$

Solution II

$$\boxed{\textbf{Option A is the correct answer}}$$

Analyzing conditions for a I

Now, we try to apply Z-Transform in this question: We cannot directly apply it in this case, due to it being non-linear. We will try to linearize this fixed-point iteration so that we can apply z-transform. Since we know $\lim_{n\to\infty} x_n = \sqrt{a}$

$$x_n = \sqrt{a} + \epsilon_n \tag{8}$$

$$x_{n+1} = g(x_n) \tag{9}$$

$$\implies \sqrt{a} + \epsilon_{n+1} = g(\sqrt{a} + \epsilon_n) \tag{10}$$

Since error is small ($\epsilon_n << 1$), we can write

$$g(\sqrt{a} + \epsilon_n) = g(\sqrt{a}) + \epsilon_n g'(\sqrt{a}) \tag{11}$$

Analyzing conditions for a II

Finding $g(\sqrt{a})$ and $g(\sqrt{a})$. From (2), (7), $g(\sqrt{a}) = \sqrt{a}$.

$$g'(x) = \frac{-6a^2\sqrt{a}}{x^3} + \frac{3a^2}{x^2} \tag{12}$$

$$g'(\sqrt{a}) = -6a + 3a = -3a \tag{13}$$

$$\epsilon_{n+1} = -3a\epsilon_n \tag{14}$$

We finally have a linear sequence. It is basically a GP. For the GP to converge and not diverge, intuitively we can say that $|-3a| < 1$. Anyways we apply Z-transform on both sides. Let E(z) be z-transform of ϵ_n.

$$zE(z) - z\epsilon_n = -3aE(z) \tag{15}$$

$$E(z) = \frac{z\epsilon_n}{z + 3a} \tag{16}$$

Analyzing conditions for a III

We find the poles:

$$z + 3a = 0 \implies z = -3a \tag{17}$$

For the iteration to be stable, and for error sequence to converge to 0, z must lie within the unit circle.

$$|z| < 1 \implies |a| < \frac{1}{3} \tag{18}$$

$$\boxed{\mathbf{x_n \text{ converges only if } |a| < 1/3}}$$

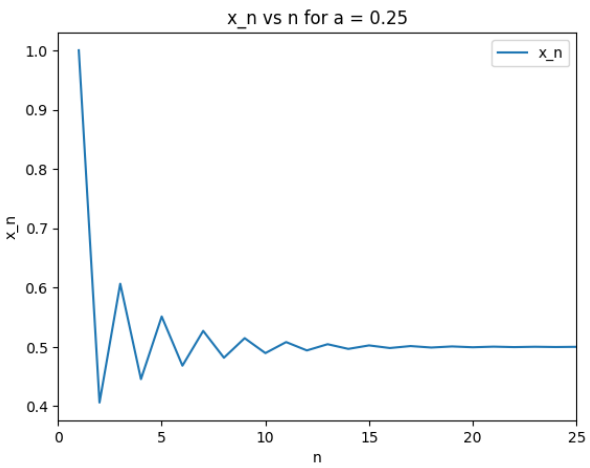
Python Code - Plotting x_n vs n I

```
import numpy as np
import matplotlib.pyplot as plt
import math

def g_x(a, x):
  return math.sqrt(a)*(1+((3*a**2)/x**2)) - 3*(a**2)/x

n=1001
print('Enter a:')
a=float(input())
x = np.arange(1, n+1)
y = np.zeros(n)
y[0] = 1
for i in range(1,n):
  y[i] = g_x(a, y[i-1])
plt.plot(x, y, label = 'x_n')
#plt.xlim(0, 25)
```

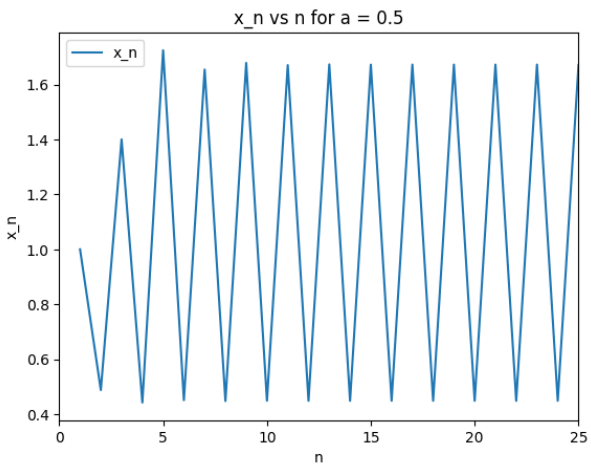
Python Code - Plotting x_n vs n II

```
plt.xlabel('n')
plt.ylabel('x_n')
plt.legend()
plt.title(f'x_n vs n for a = {a}')
print(f'x_n after {n-1} iterations: x_{n-1} = {y[n-1]}')
```

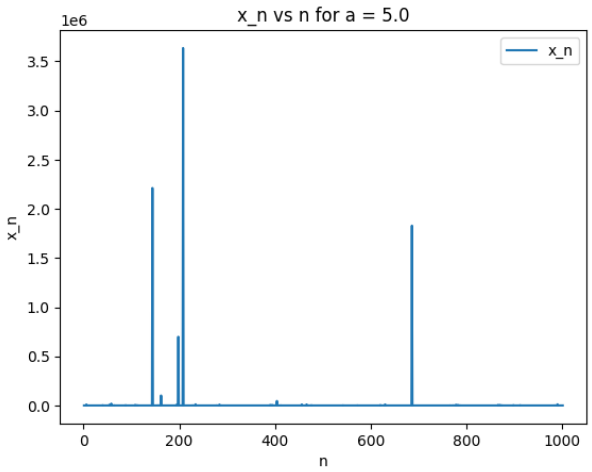
Plot for a=0.25, a=0.5, a=5



(a) $a = 0.25$



(b) $a = 0.5$



(c) $a = 5$

Python Code - Computing and plotting log of absolute error vs n I

```
import numpy as np
import matplotlib.pyplot as plt
import math

def g_x(a, x):
  return math.sqrt(a)*(1+((3*a**2)/x**2)) - 3*(a**2)/x

n=120
print('Enter a:')
a=float(input())
x = np.arange(1, n+1)
y = np.zeros(n)
y[0] = 1
for i in range(1,n):
  y[i] = g_x(a, y[i-1])
z = np.zeros(n)
```

Python Code - Computing and plotting log of absolute error vs n II

```
for i in range(1, n):
  z[i] = np.log(np.abs(y[i] - y[n-1]))
plt.plot(x, z, label = 'x_n')
plt.xlabel('n')
plt.ylabel(f'log(x_n-{math.sqrt(a)})')
plt.legend()
plt.title(f'log_10(x_n-{math.sqrt(a)}) vs n for a = {a}')
print(f'log_10(x_n-{math.sqrt(a)}) after {n-1} iterations: {z[n
    -1]}')
```

Plot for log absolute error of x_n

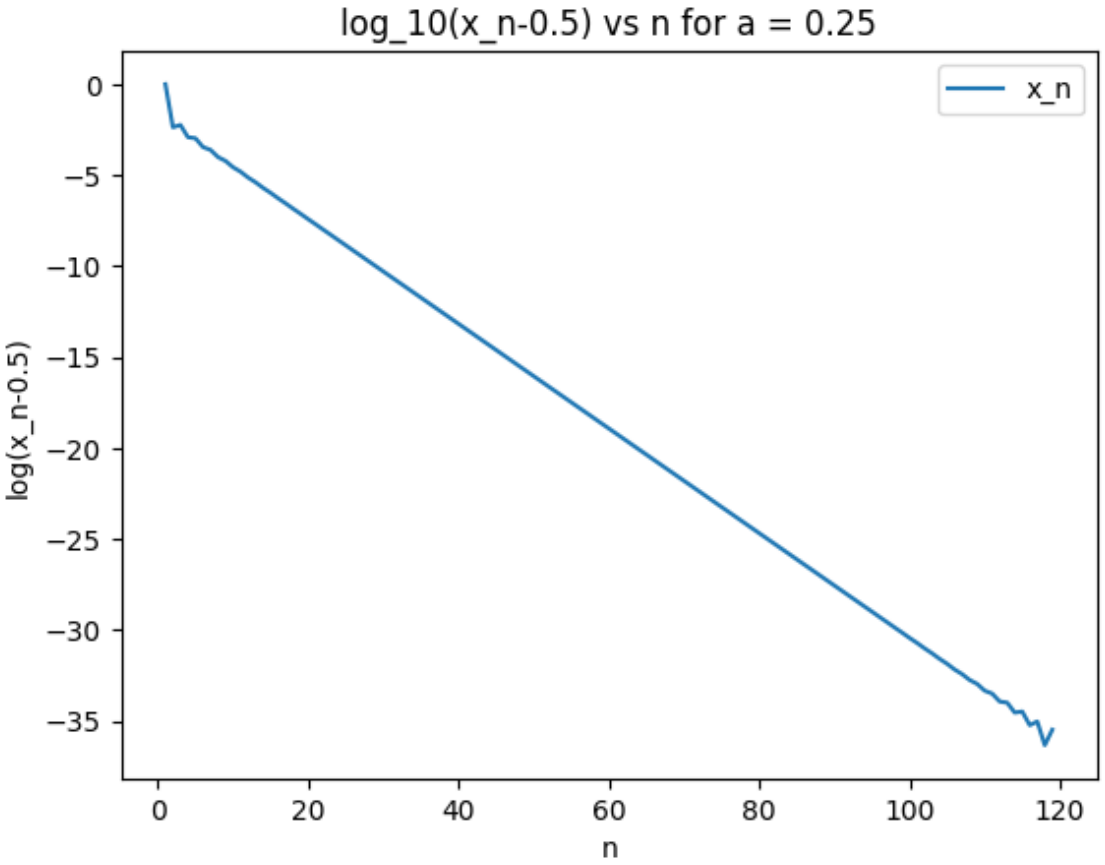


Figure: Plot for log absolute error

Python Code - Plotting cobweb plot I

```
import numpy as np
import matplotlib.pyplot as plt
import math

def g_x(a, x):
  return math.sqrt(a)*(1+((3*a**2)/x**2)) - 3*(a**2)/x

print('Enter a:')
a = float(input())
x0 = 1.0
n_iterations = 10
true_limit = math.sqrt(a)

x_vals = np.linspace(0.1, 2, 500)
y_vals = g_x(a, x_vals)
```

Python Code - Plotting cobweb plot II

```
plt.plot(x_vals, x_vals, '--', color='gray', label='y = x (
    Diagonal)')
plt.plot(x_vals, y_vals, color='blue', linewidth=2, label='y = g(
    x)')

cobweb_x = [x0]
cobweb_y = [0]

current_x = x0

for i in range(n_iterations):
    next_x = g_x(a, current_x)

    cobweb_x.append(current_x)
    cobweb_y.append(next_x)

    cobweb_x.append(next_x)
```

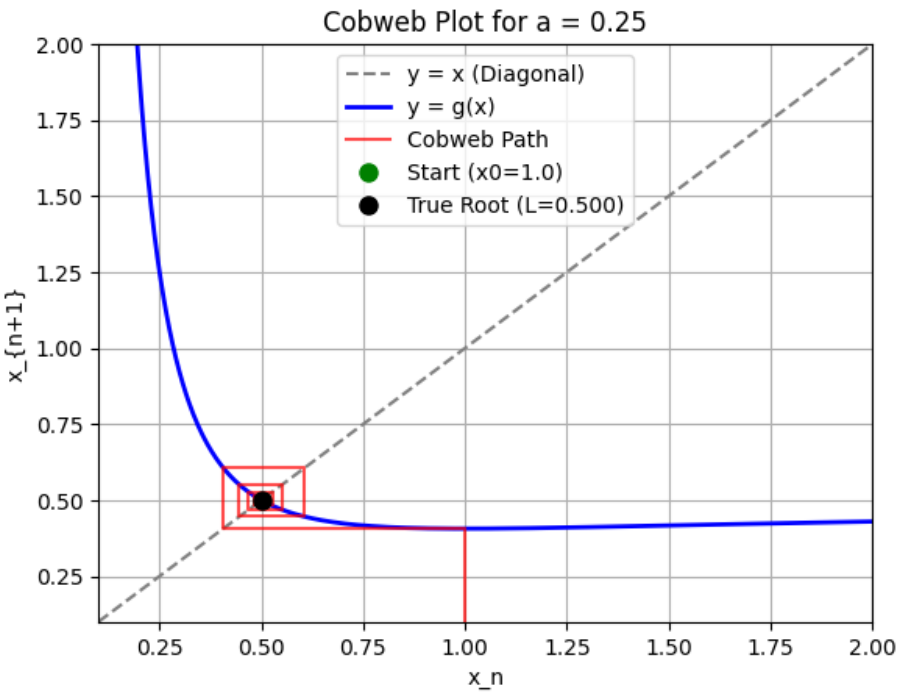
Python Code - Plotting cobweb plot III

```
    cobweb_y.append(next_x)

    current_x = next_x

plt.plot(cobweb_x, cobweb_y, color='red', alpha=0.7, linewidth
    =1.5, label='Cobweb Path')

plt.plot(x0, 0, 'go', markersize=8, label=f'Start (x0={x0})')
plt.plot(true_limit, true_limit, 'ko', markersize=8, label=f'True
     Root (L={true_limit:.3f})')

plt.xlim(0.1, 2)
plt.ylim(0.1, 2)
plt.xlabel('x_n')
plt.ylabel('x_{n+1}')
plt.title(f'Cobweb Plot for a = {a}')
plt.grid(True)
```

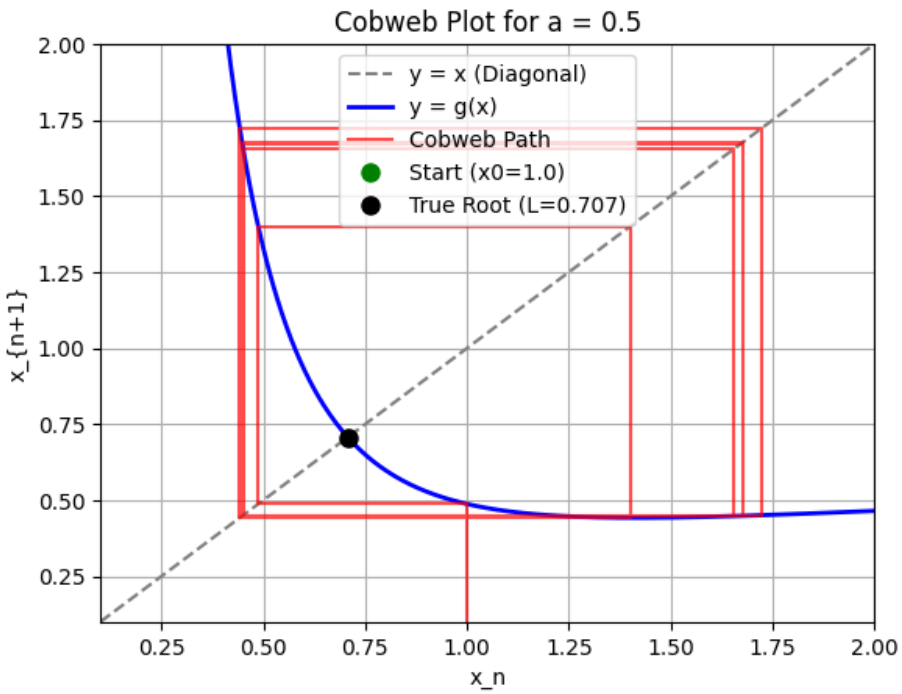
Python Code - Plotting cobweb plot IV

```
plt.legend()
plt.show()
```

Cobweb Plot for $a < \frac{1}{3}$ and $a > \frac{1}{3}$



(a) $a = 0.25 < 0.33$



(b) $a = 0.5 > 0.33$